

Ecole Hassania des Travaux Publics
Executive Master Data Engineering (MSDE) — Edition 7

Projet du Module 5 : Machine Learning

Prédiction du Prix de Vente d'une Voiture d'Occasion

Étudiant	BEN-EL-AHMAR BADR
Encadrant	Pr. Abdelhamid Fadil
Dataset	Used Cars
Taille du dataset	371 528 lignes × 21 colonnes
Variable cible	price (prix de vente)
Modèle final	XGBoost (après tuning RandomizedSearchCV)
Dépôt GitHub	github.com/badrntest-creator/Project-ML- -Used-Car-Price-Prediction
Application déployée	project-ml-used-car-price-prediction.streamlit.app

Table des matières

Table des matières	2
I. Compréhension de la Problématique	3
Problème métier	3
Traduction en tâche ML	3
État de l'art	3
II. Analyse Exploratoire des Données (EDA)	4
Présentation du dataset	4
Valeurs manquantes	4
Variable cible et distribution	4
Observations principales	4
Outliers	5
III. Pré-traitement des Données (Preprocessing)	6
Features sélectionnées	6
IV. Construction et Évaluation des Modèles ML	7
V. Tuning des Modèles Candidats	8
Hyperparamètres explorés	8
Comparaison avant / après tuning	8
VI. Modèle Final et Sérialisation	9
Sélection du modèle final	9
Courbes d'apprentissage	9
Prédictions vs valeurs réelles	9
Sérialisation du pipeline	10
VII. Déploiement de l'Application Streamlit	10
Architecture de l'application	10
Structure du dépôt GitHub	11
Liens du projet	11
VIII. Conclusion	11
Bilan du projet	12

I. Compréhension de la Problématique

Problème métier

Le marché des voitures d'occasion est caractérisé par une forte asymétrie d'information entre acheteurs et vendeurs. Estimer le juste prix d'un véhicule requiert une analyse multi-dimensionnelle combinant l'âge, le kilométrage, la marque, le type de carburant et de nombreuses autres caractéristiques. Le Machine Learning offre une approche data-driven permettant d'apprendre ces patterns à partir de données historiques pour produire des estimations objectives et reproductibles.

Le dataset utilisé est issu de la plateforme Kaggle et contient environ 370 000 annonces de voitures d'occasion avec leurs caractéristiques techniques et commerciales. La variable cible est **price** (prix de vente en euros).

Traduction en tâche ML

Il s'agit d'un problème de **régression supervisée** : la variable cible **price** est numérique continue. L'objectif est de construire un **modèle $f(X) = \text{price}$** minimisant l'erreur de prédiction.

État de l'art

La littérature privilégie les méthodes ensemblistes (Random Forest, XGBoost, LightGBM...), qui surpassent les modèles linéaires grâce à leur capacité à capturer les relations non linéaires entre les caractéristiques et le prix.

Algorithme	Type	Avantage principal
Régression Linéaire	Linéaire	Baseline interprétable
Ridge / Lasso	Linéaire régularisée	Gère la multicollinéarité
Arbre de Décision	Arbre	Non linéaire, interprétable
Random Forest	Ensemble (Bagging)	Robuste, gère les valeurs manquantes
Gradient Boosting	Ensemble (Boosting)	Haute précision
XGBoost	Ensemble (Boosting)	Rapide et régularisé
LightGBM	Ensemble (Boosting)	Très rapide sur grandes données
SVR	Noyau	Efficace en haute dimension
KNN Regressor	Instance	Simple, sans hypothèses
Extra Trees	Ensemble (Bagging)	Variance réduite par randomisation

II. Analyse Exploratoire des Données (EDA)

Présentation du dataset

Le dataset contient **371 528 lignes et 21 colonnes**. Colonnes principales : name, brand, model, vehicleType, yearOfRegistration, gearbox, powerPS, kilometer, fuelType, notRepairedDamage et price.

Valeurs manquantes

Plusieurs colonnes présentent des valeurs manquantes, notamment notRepairedDamage (19,4 %), vehicleType (10,2 %), fuelType (9 %), model (5,5 %) et gearbox (5,4 %). Ces colonnes nécessitent une stratégie d'imputation au preprocessing.

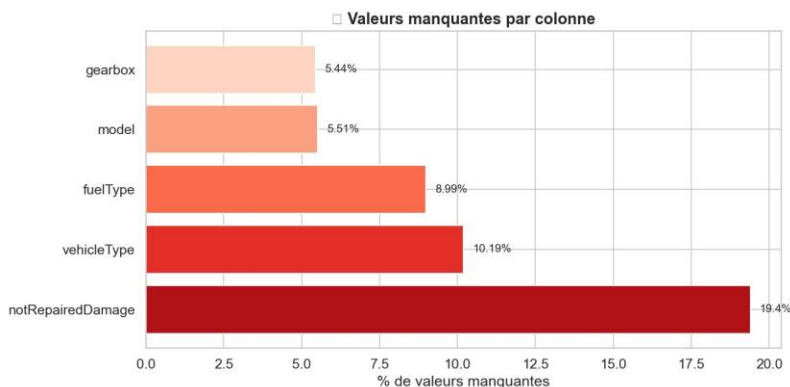


Figure 1 — Pourcentage de valeurs manquantes par colonne

Variable cible et distribution

La variable price présente une distribution fortement asymétrique à droite, concentrée entre 1 000 et 20 000 €, avec quelques valeurs extrêmes jusqu'à 100 000 €. Une transformation logarithmique $\log(1+price)$ est appliquée pour normaliser la distribution.

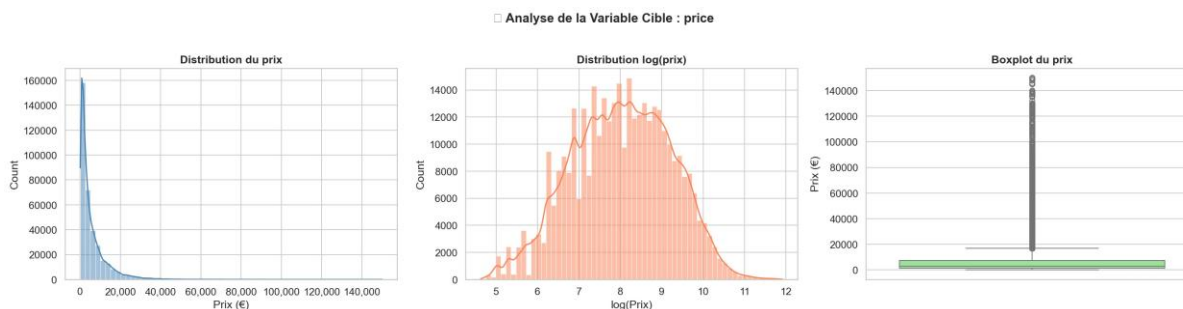


Figure 2 — Distribution de la variable cible price (brute, log-transformée, boxplot)

Observations principales

- Marques premium (BMW, Mercedes, Porsche) : prix médians nettement plus élevés.
- Boîtes automatiques associées à des prix plus élevés.
- Véhicules sans dommages non réparés : prix significativement plus élevé.
- Année d'immatriculation : corrélation positive avec le prix (plus récent = plus cher).
- Puissance (powerPS) : corrélation positive avec le prix.

- Kilométrage : corrélation négative avec le prix.

Outliers

La méthode IQR révèle de nombreux outliers sur price, yearOfRegistration et powerPS. Seuils de filtrage retenus : price entre 100 et 100 000 €, yearOfRegistration entre 1950 et 2024, powerPS entre 10 et 500 PS.

III. Pré-traitement des Données (Preprocessing)

Chaque étape de préparation est justifiée par les observations de l'EDA et les exigences des algorithmes ML.

Étape	Action	Justification
1. Suppression colonnes	dateCrawled, lastSeen, abtest...	Non pertinentes pour la prédiction
2. Filtrage outliers	price, yearOfRegistration, powerPS, kilometer	Valeurs aberrantes faussant le modèle
3. Imputation	Mode pour les variables catégorielles	Préserve la distribution originale
4. Feature Engineering	car_age = 2024 - yearOfRegistration ; log_price = log(1+price)	Améliore la linéarité et la pertinence
5. Encodage	LabelEncoder sur 6 colonnes	Conversion numérique requise par sklearn
6. Normalisation	StandardScaler sur 4 features numériques	Nécessaire pour Ridge, Lasso, KNN
7. Split	80 % train / 20 % test, random_state=42	Reproductibilité et évaluation fiable

Features sélectionnées

Feature	Type	Description
car_age	Numérique (scale)	Âge du véhicule = 2024 - année
powerPS	Numérique (scale)	Puissance en chevaux
kilometer	Numérique (scale)	Kilométrage
monthOfRegistration	Numérique (scale)	Mois d'immatriculation
vehicleType_encoded	Catégorielle (encodée)	Type de carrosserie
gearbox_encoded	Catégorielle (encodée)	Boîte de vitesses
fuelType_encoded	Catégorielle (encodée)	Type de carburant
notRepairedDamage_encoded	Catégorielle (encodée)	Dommages non réparés
brand_encoded	Catégorielle (encodée)	Marque
model_encoded	Catégorielle (encodée)	Modèle

IV. Construction et Évaluation des Modèles ML

11 algorithmes de régression ont été entraînés et évalués sur le jeu de test.

Modèle	R ²	RMSE	MAPE (%)	CV R ²
Random Forest	0.8582	0.4435	38.58	0.8533
XGBoost	0.8576	0.4443	40.42	0.8586
Extra Trees	0.8484	0.4585	39.13	0.8485
LightGBM	0.8485	0.4583	41.96	0.8497
Gradient Boosting	0.8224	0.4962	46.08	-
KNN Regressor	0.8227	0.4958	45.84	-
Arbre de Décision	0.7785	0.5542	48.30	-
Régression Linéaire	0.6271	0.7191	82.54	-
Ridge	0.6271	0.7191	82.54	-
Lasso	0.6270	0.7191	82.58	-
ElasticNet	0.6270	0.7191	82.55	-

Les 4 premières lignes correspondent aux modèles retenus pour le tuning.

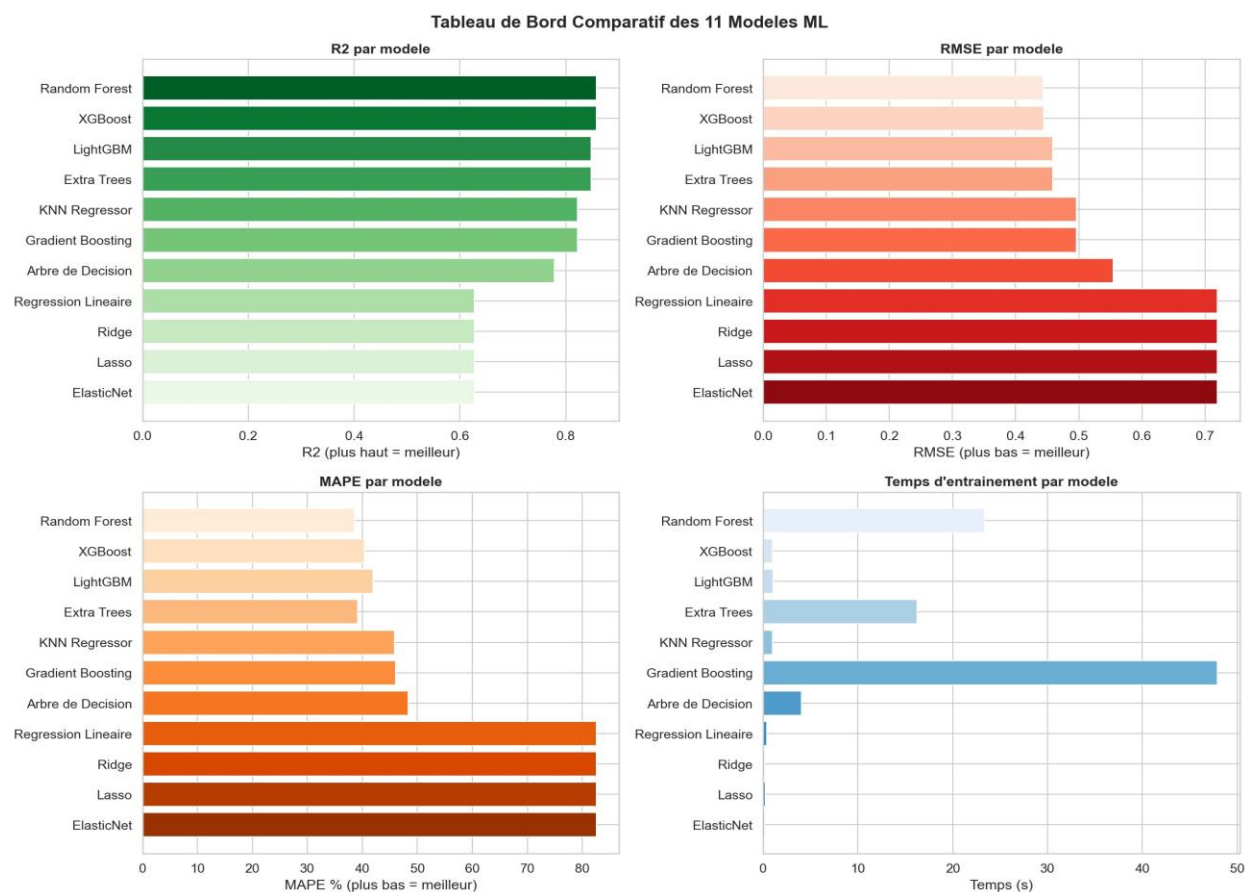


Figure 3 — Tableau de bord comparatif : R², RMSE, MAPE et temps d'entraînement

Les modèles ensemblistes (Random Forest, XGBoost, Extra Trees, LightGBM) surpassent nettement les modèles linéaires, confirmant une relation fortement non linéaire entre les caractéristiques du véhicule et son prix. L'écart entre R^2 test et CV R^2 reste minimal pour ces modèles, sans overfitting significatif.

V. Tuning des Modèles Candidats

Les 3 meilleurs modèles (Random Forest, XGBoost, LightGBM) ont été optimisés par RandomizedSearchCV avec validation croisée 3-fold ($n_iter=20$), critère de scoring : R^2 .

Hyperparamètres explorés

Modèle	Hyperparamètres explorés
Random Forest	<code>n_estimators</code> , <code>max_depth</code> , <code>min_samples_split</code> , <code>min_samples_leaf</code> , <code>max_features</code>
XGBoost	<code>n_estimators</code> , <code>max_depth</code> , <code>learning_rate</code> , <code>subsample</code> , <code>colsample_bytree</code> , <code>min_child_weight</code>
LightGBM	<code>n_estimators</code> , <code>num_leaves</code> , <code>learning_rate</code> , <code>min_child_samples</code> , <code>reg_alpha</code> , <code>reg_lambda</code> , <code>subsample</code>

Comparaison avant / après tuning

Modèle	Phase	R^2	RMSE	MAPE (%)	CV R^2
LightGBM	Avant Tuning	0.8485	0.4583	41.96	0.8497
LightGBM	Après Tuning	0.8659	0.4313	39.00	0.8662
Random Forest	Avant Tuning	0.8582	0.4435	38.58	0.8533
Random Forest	Après Tuning	0.8678	0.4281	37.53	0.8647
XGBoost	Avant Tuning	0.8576	0.4443	40.42	0.8586
XGBoost	Après Tuning	0.8679	0.4279	38.19	0.8679

Le tuning apporte un gain pour les 3 modèles. XGBoost et Random Forest atteignent un R^2 de 0.8679 après optimisation, avec un RMSE respectif de 0.4279 et 0.4281.

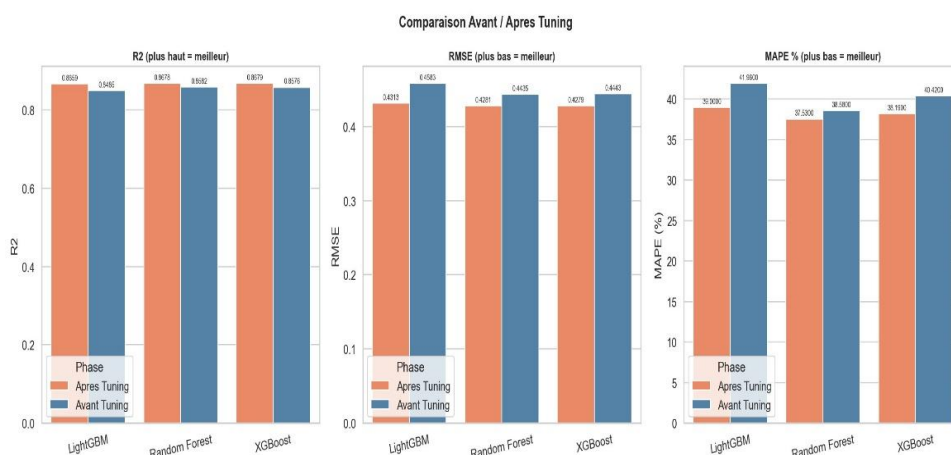


Figure 4 — Comparaison des performances avant et après tuning

VI. Modèle Final et Sérialisation

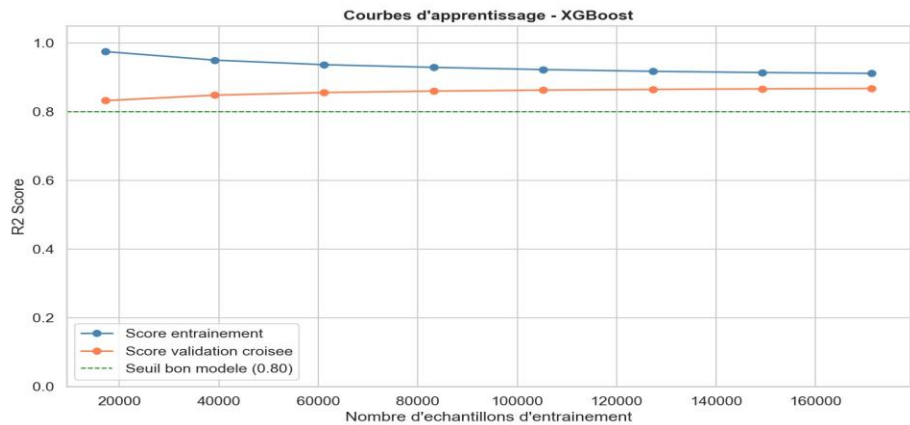
Sélection du modèle final

Le modèle final retenu est XGBoost après tuning, choisi pour : R^2 le plus élevé (0.8679), RMSE le plus faible (0.4279), écart minimal entre R^2 test et CV R^2 (absence d'overfitting), et temps d'entraînement raisonnable pour la production.

Métrique	Valeur	Interprétation
R^2 (test)	0.8679	86,79 % de la variance expliquée
RMSE (log)	0.4279	Erreur quadratique moyenne sur log(price)
MAE (log)	0.4181	Erreur absolue moyenne sur log(price)
MAPE (%)	38.19 %	Erreur moyenne de 38 % sur le prix réel
CV R^2 (3-fold)	0.8679	Bonne généralisation, pas d'overfitting

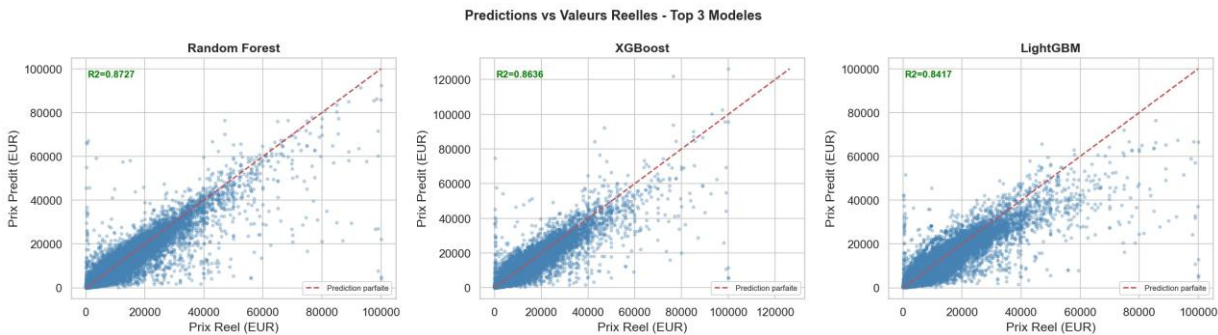
Courbes d'apprentissage

Les courbes d'apprentissage confirment l'absence d'overfitting : les scores d'entraînement et de validation convergent vers une valeur élevée de R^2 , indiquant une bonne généralisation.



Prédictions vs valeurs réelles

Les points sont bien alignés autour de la droite $y = x$, confirmant la qualité des prédictions sur le jeu de test.



Sérialisation du pipeline

Le pipeline final intègre le préprocesseur (StandardScaler) et le modèle XGBoost optimisé dans un objet sklearn Pipeline unique, sérialisé avec joblib sous le nom pipeline_final.pkl (7,8 MB). Cela garantit que les mêmes transformations seront appliquées à l'entraînement et en production.

VII. Déploiement de l'Application Streamlit

Architecture de l'application

L'application permet à l'utilisateur de saisir les caractéristiques d'un véhicule et d'obtenir instantanément une estimation du prix de vente. L'interface est divisée en deux colonnes : formulaire de saisie à gauche, résultat de la prédiction à droite.

Fonctionnalité	Description
Saisie des features	Marque, modèle dynamique, type, année, km, puissance, carburant, boîte
Menus dynamiques	Les modèles disponibles changent selon la marque sélectionnée
Affichage prédiction	Prix estimé en euros avec fourchette basse/haute ($\pm 15\%$)
Niveau de confiance	Jauge visuelle avec label Haute / Moyenne / Faible
Récapitulatif	Tableau des caractéristiques saisies par l'utilisateur
Déploiement	Streamlit Cloud avec URL publique



Figure 7 — Application Streamlit : interface de prédiction

Structure du dépôt GitHub

Fichier / Dossier	Description
notebooks/01_EDA.ipynb	Analyse exploratoire des données
notebooks/02_Preprocessing.ipynb	Pré-traitement et feature engineering
notebooks/03_Modeling.ipynb	Construction et évaluation des 11 modèles
notebooks/04_Tuning.ipynb	Tuning et sélection du modèle final
models/pipeline_final.pkl	Pipeline ML sérialisé (XGBoost + préprocesseur)
models/encoders.pkl	Encodeurs LabelEncoder sérialisés
app/app.py	Application Streamlit de prédiction
requirements.txt	Dépendances Python du projet
README.md	Documentation du projet avec liens GitHub et Streamlit

Liens du projet

- Dépôt GitHub : <https://github.com/badrntest-creator/Project-ML--Used-Car-Price-Prediction>
- Application Streamlit : <https://project-ml-used-car-price-prediction.streamlit.app/>

The screenshot shows the GitHub repository interface. At the top, there's a navigation bar with links like Platform, Solutions, Resources, Open Source, Enterprise, and Pricing. Below this, the repository name 'badrntest-creator / Project-ML--Used-Car-Price-Prediction' is displayed, along with a 'Public' label and buttons for Notifications, Fork (0), and Star (0). A secondary navigation bar includes links for Code, Issues, Pull requests, Actions, Projects, Security and quality, and Insights. The main content area shows the file structure of the repository, including folders like 'app', 'data', 'models', 'notebooks', and 'report', and a file 'requirements.txt'. Each item is accompanied by a brief description and a timestamp. On the right side, there's an 'About' section with a placeholder for a description, and sections for 'Releases' and 'Packages', both indicating that no releases or packages have been published yet.

File / Folder	Description	Time
app	Streamlit App	1 hour ago
data	Data Xy_train/test	1 hour ago
models	ML models	1 hour ago
notebooks	Delete notebooks/app directory	1 hour ago
report	Report Graphes.png	1 hour ago
requirements.txt	Add files via upload	1 hour ago

VIII. Conclusion

Ce projet a permis de mettre en œuvre un **pipeline Machine Learning complet** appliqué à la **prédiction du prix de vente de voitures d'occasion**. À partir d'un dataset de **370 000** annonces (**Kaggle** / eBay Kleinanzeigen), une analyse exploratoire rigoureuse et un preprocessing adapté ont été réalisés, 11 algorithmes de régression ont été testés, puis les 3 meilleurs ont été optimisés par RandomizedSearchCV.

Le modèle final retenu est **XGBoost** après tuning, atteignant un **R^2 de 0.8679** sur le jeu de test avec un **RMSE de 0.4279** (sur **log_price**). Il a été sérialisé dans un pipeline sklearn complet et déployé dans une **application Streamlit** interactive accessible publiquement.

Bilan du projet

Axe	Réalisations
1. Compréhension problématique	Problème métier défini, tâche ML identifiée, état de l'art
2. EDA	Distributions, corrélations, outliers, visualisations
3. Preprocessing	Filtrage, imputation, feature engineering, encodage, normalisation
4. Construction modèles ML	11 algorithmes testés avec métriques et graphes de performance
5. Tuning + modèle final	RandomizedSearchCV sur 3 candidats, comparaison avant/après, courbes d'apprentissage
6. Déploiement	Streamlit Cloud + dépôt GitHub structuré + pipeline sérialisé
7. Présentation	Rapport structuré, commentaires synthétiques, application ergonomique